

Cerințe obligatorii

1. Pattern-urile implementate trebuie să respecte definiția din GoF discutată în cadrul cursurilor și laboratoarelor. Nu sunt acceptate variații sau implementări incomplete.
2. Pattern-ul trebuie implementat în totalitate corect pentru a fi luat în calcul.
3. Soluția nu conține erori de compilare.
4. Pattern-urile pot fi tratate distinct sau pot fi implementate pe același set de clase.
5. Implementările care nu au legătura funcțională cu cerințele din subiect NU vor fi luate în calcul (preluare unui exemplu din alte surse nu va fi punctată).
6. NU este permisă modificarea claselor primite.
7. Soluțiile vor fi verificate încrucișat folosind MOSS. Nu este permisă partajarea de cod între studenți. Soluțiile care au un grad de similitudine mai mare de 30% vor fi anulate.

Cerințe Clean Code obligatorii (soluția este depunctată cu câte 2 puncte pentru fiecare cerință ce nu este respectată) - maxim se pot pierde 8 puncte

1. Pentru denumirea claselor, funcțiilor, testelor unitare, atributelor și a variabilelor se respecta convenția de nume de tip Java Mix CamelCase;
2. Pattern-urile și clasa ce conține metoda main() sunt definite în pachete distincte ce au forma *cts.nume.prenume.gNrGrupa.denumire_pattern*, *cts.nume.prenume.gNrGrupa.main* (studenții din anul suplimentar trec "as" în loc de gNrGrupa);
3. Clasele și metodele sunt implementate respectând principiile KISS, DRY și SOLID (atenție la DIP);
4. Denumirile de clase, metode, variabile, precum și mesajele afișate la consola trebuie să aibă legătura cu subiectul primit (nu sunt acceptate denumiri generice). Funcțional, metodele vor afișa mesaje la consola care să simuleze acțiunea cerută sau vor implementa prelucrări simple.

Se dezvoltă o aplicație software destinată unui magazin online.

- 5p.** În cadrul unui magazin online, există un modul de autentificare a clienților pe baza unei adrese de email și a unei parole. Însa, s-a observat că există situații în care, pentru anumite adrese de email, se încearcă repetat introducerea de parole diferite în scopul autentificării. Astfel, s-a luat decizia introducerii unui modul securizat de autentificare care să blocheze un cont al unui client care încearcă să se autentifice cu un număr de 5 parole gresite în decurs de 1 oră. De asemenea, trebuie introdus un mecanism de resetare număr încercări eșuate. Implementarea securizată nu trebuie să implice modificări la nivelul actualului modul de autentificare. (se vor folosi interfețe *IClient*, *IMagazin*, *IAutentificare*).
- 3p.** Pattern-ul este testat în *main()* prin popularea magazinului cu un număr minim de 3 clienți. Ulterior, se va testa procesul de autentificare prin lansarea de cereri de autentificare până când unui client i se blochează contul.
- 1p.** Să se motiveze în scris alegerea design pattern-ului care modelează cel mai bine situația enunțată. Motivarea trebuie să conțină elementele cheie care denotă utilizarea respectivului design pattern.
- 5p.** De asemenea, în cadrul magazinului online, se dorește adăugarea unei facilități acordată clienților. Această facilitate trebuie să permită clienților posibilitatea de a reveni într-o stare anterioară a unei comenzi reținută pentru client, tocmai pentru a facilita cumpărăturile recurente. Astfel, fiecare client poate stoca o singură versiune de cos de cumpărături, precum și resetarea cosului curent.

de cumparaturi pe baza cosului anterior salvat. Sa se aleaga si sa se implementeze acel design pattern care modeleaza cel mai bine situatia enuntata. (se va folosi interfata IClient)

- 3p.** Pattern-ul este testat în main() prin definirea unui client, crearea unui cos de cumparaturi, salvarea sa, crearea altui cos de cumparaturi si restaurarea cosului curent cu unul anterior salvat.
- 1p.** Sa se motiveze in scris alegerea design pattern-ului care modeleaza cel mai bine situatia enuntata. Motivarea trebuie sa contina elementele cheie care denota utilizarea respectivului design pattern.